M01 — Fundamentos de ASP.NET Core MVC

Cuaderno de prácticas

Índice del módulo

- **P1.1. Primera aplicación MVC y recorrido de una petición HTTP**
- **P1.2. El flujo Modelo–Vista–Controlador**
- **P1.3. Estructura y convenciones del proyecto**
- **P1.4. Controladores, acciones, parámetros y rutas**
- **P1.5. Vistas tipadas, layout, parciales y navegación**

P1.1. Primera aplicación MVC y recorrido de una petición HTTP

A. Identificación

Objetivo. Crear una aplicación MVC a partir de la plantilla de .NET 8, ejecutar el proyecto inicial suministrado en el curso, observar una petición real y distinguir los cambios en las cabeceras HTTP de los cambios en el HTML generado por una vista Razor.

Teoría relacionada. Punto 1.1, especialmente flujo HTTP, diferencia MVC/API, papel de Kestrel, `Program.cs`, acción MVC, vista Razor y cabeceras HTTP.

Conocimientos previos. C# básico, uso de una terminal, HTTP GET y experiencias anteriores con Web API. No se requiere conocer Razor ni su sintaxis en profundidad.

Producto de esta práctica. Una aplicación MVC que responde en `http://localhost:5091/`, cuyo estado inicial ya está dispuesto en el repositorio y cuyos dos pequeños cambios pueden activarse de forma independiente.

B. Preparación

Herramientas y dependencias

Se necesita .NET SDK 8, navegador, editor C# (preferentemente Visual Studio con soporte ASP.NET Core) y Git. La práctica se describe con comandos de la CLI de .NET, equivalentes al botón de ejecución de Visual Studio. Para inspeccionar HTTP usaremos la pestaña *Red/Network* del navegador; como comprobación alternativa puede usarse `curl.exe` en Windows o `curl` en Linux/macOS.

Comprobar el SDK:

```
dotnet --list-sdks
```

Resultado esperado: al menos una entrada cuya versión comience por `8.0.`. Una versión posterior no sustituye necesariamente al SDK 8 en la ejecución de esta práctica, cuyo proyecto apunta explícitamente a `net8.0`.

Obtener el estado inicial exacto

En una carpeta de trabajo elegida por el alumno:

```
git clone --branch desarrollo/m01-mvc https://github.com/jaimecopilot/CURSO-DESARROLLO-WEB-NET8.git
cd CURSO-DESARROLLO-WEB-NET8
```

El proyecto del capítulo está ubicado en:

```
M01_MVC/
  PROYECTO/
    1.1/
      CursoWeb.Mvc/
        CursoWeb.Mvc.csproj
        Program.cs
        Controllers/
          HomeController.cs
        Views/
          _ViewImports.cshtml
```

```
    _ViewStart.cshtml
    Home/
      Index.cshtml
    Shared/
      _Layout.cshtml
  wwwroot/
    css/
      site.css
  appsettings.json
  Properties/
    launchSettings.json
```

Este árbol identifica **todos los archivos necesarios** para el primer recorrido. No necesita base de datos, credenciales ni servicios externos. La práctica no depende del código de ningún otro curso.

Estado inicial para los bloques activables: la cabecera `X-Curso-Paso` está comentada en el controlador y el bloque `marca-paso-1-1-5` está dentro de un comentario Razor. Por tanto, la página se muestra, pero aún no incorpora las dos variaciones.

C. Desarrollo paso a paso

Paso 1.1.1 — Crear una aplicación ASP.NET Core MVC desde la plantilla

Primero generaremos una aplicación de prueba para identificar la plantilla oficial sin modificar el proyecto docente.

Estando en la **raíz del repositorio**, ejecutar:

```
dotnet new mvc -f net8.0 --no-https -n PlantillaMvc -o .work/PlantillaMvc
```

Qué hace cada opción:

- `dotnet new mvc` utiliza la plantilla de aplicación web MVC instalada con el SDK.
- `-f net8.0` establece el framework de destino.
- `--no-https` simplifica exclusivamente esta primera prueba HTTP local; no se trata de una recomendación de producción.
- `-n PlantillaMvc` nombra el proyecto temporal.
- `-o .work/PlantillaMvc` coloca los archivos en una carpeta descartable excluida de Git.

Para ver que la plantilla es ejecutable:

```
dotnet run --project .work/PlantillaMvc/PlantillaMvc.csproj --no-launch-profile --urls http://
localhost:5095
```

Abrir `http://localhost:5095/`. Debe aparecer la página inicial de la plantilla MVC, con su navegación y su diseño estándar. Detener el proceso con **Ctrl+C**.

Qué acabamos de comprobar: la plantilla genera una aplicación con código servidor, vistas Razor y recursos estáticos. No se ha creado una API REST; se ha creado una web que produce HTML.

El resto de la práctica usará el proyecto **CursoWeb.Mvc** del repositorio, de contenido controlado y vinculado exactamente con este cuaderno. No se debe sustituir ese proyecto por `PlantillaMvc`, porque contienen vistas y nombres distintos.

Paso 1.1.2 — Abrir y ejecutar el proyecto del curso

Desde la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.1/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Alternativa Visual Studio: abrir `CursoWeb.Mvc.csproj`, seleccionar el perfil de ejecución local `CursoWeb.Mvc` (HTTP) y ejecutar. Comprobar el puerto mostrado en la salida. Si se utiliza otro puerto desde Visual Studio, todas las URL siguientes deben adaptarse a ese valor.

Esperar a que la consola indique que el servidor escucha en `http://localhost:5091`. Abrir esa dirección en el navegador.

Resultado esperado visible:

- Un enlace `Inicio` en la cabecera.
- El título `1.1 | Primer recorrido MVC`.
- El texto `Esta página HTML ha sido generada en el servidor mediante una acción MVC.`.
- El mensaje `Respuesta HTTP correcta: aplicación MVC disponible.`.
- Un pie con `Curso ASP.NET Core MVC · .NET 8`.

No debe aparecer todavía `Contenido añadido durante la práctica`. Si aparece, probablemente el bloque del paso 1.1.5 ya esté activado.

Qué ocurre internamente: Kestrel recibe la petición a `/`. La ruta predeterminada selecciona `HomeController.Index()`. La acción devuelve `View()` y MVC renderiza la vista `Views/Home/Index.cshtml`, rodeada por `Views/Shared/_Layout.cshtml`. El navegador recibe el HTML y solicita después la hoja de estilos. Es el primer recorrido extremo a extremo de la aplicación.

Paso 1.1.3 — Inspeccionar petición, ruta y archivos responsables

Con la página abierta, pulsar **F12** (o abrir las herramientas para desarrolladores del navegador), seleccionar **Red/Network** y recargar la página. Si no aparece ninguna fila, mantener abierta la pestaña de red y volver a recargar.

Seleccionar la solicitud de tipo **document** cuyo destino sea `http://localhost:5091/`. Identificar:

1. **Request URL:** `http://localhost:5091/`.
2. **Request Method:** `GET`.
3. **Status Code:** `200`.
4. **Response Headers:** una cabecera `Content-Type` cuyo valor comienza por `text/html`.
5. **Response/Respuesta:** el cuerpo HTML; buscar el texto `Primer recorrido MVC`.

También aparecerá una petición a `/css/site.css`, con tipo de contenido CSS. Esa solicitud es otro recorrido HTTP: el recurso físico está bajo `wwwroot/css/site.css` y lo puede devolver el middleware de archivos estáticos.

Ahora abrir en el editor los cuatro archivos siguientes, **sin modificarlos todavía**:

- `Program.cs`: localizar `AddControllersWithViews()`, `UseStaticFiles()` y `MapControllerRoute(...)`.
- `Controllers/HomeController.cs`: localizar la acción `Index()` y su `return View();`.
- `Views/Home/Index.cshtml`: localizar el `<h1>` que aparece en el navegador.
- `Views/Shared/_Layout.cshtml`: localizar la navegación y `@RenderBody()`.

Seguir mentalmente el recorrido **URL → ruta → controlador → vista → layout → HTML**. Anotar qué archivo determina la URL y qué archivo contiene el texto visible. Es importante diferenciar esos dos conceptos antes de añadir más acciones en 1.4.

Comprobación HTTP alternativa (manteniendo abierta la aplicación, en una segunda terminal):

```
curl -i http://localhost:5091/
```

En Windows PowerShell también puede utilizarse `curl.exe -i http://localhost:5091/`. La salida debe mostrar una línea de estado `200 OK`, cabeceras y HTML. Las cabeceras concretas pueden variar según sistema y servidor; no se exige un orden determinado.

Paso 1.1.4 — Activar una cabecera HTTP desde la acción

Dejar de ejecutar la aplicación mediante **Ctrl+C** antes de modificar y volver a construir. Abrir `Controllers/HomeController.cs` y localizar:

```
// PASO 1.1.4 — Descomentar solo la siguiente línea para observar
// una cabecera de respuesta HTTP generada desde el controlador.
// Response.Headers["X-Curso-Paso"] = "1.1.4";
```

Quitar las dos barras `//` **solo** delante de la asignación; las líneas explicativas permanecen comentadas. El fragmento activo quedará así:

```
Response.Headers["X-Curso-Paso"] = "1.1.4";
return View();
```

Guardar y ejecutar de nuevo:

```
dotnet run --no-launch-profile --urls http://localhost:5091
```

Recargar la página y observar las cabeceras de **respuesta** en F12 → Red. Debe aparecer:

```
X-Curso-Paso: 1.1.4
```

También se puede comprobar con `curl -i http://localhost:5091/`.

Qué hemos aprendido: la cabecera se añade desde el controlador a la respuesta HTTP, pero no altera el título ni el texto HTML. Si solo miramos la pantalla, podemos pensar incorrectamente que el cambio no se ha aplicado.

Comprobación negativa: la cabecera no tiene por qué aparecer en la petición CSS, porque ese archivo estático no lo sirve la acción `Index()`. La respuesta de la página y la respuesta de la hoja de estilos proceden de solicitudes diferentes.

Paso 1.1.5 — Activar un bloque HTML de la vista Razor

Detener de nuevo la aplicación. Abrir `Views/Home/Index.cshtml`. Al final aparece este fragmento:

```
@* PASO 1.1.5 (inicio)
<section id="marca-paso-1-1-5">
    <h2>Contenido añadido durante la práctica</h2>
    <p>El navegador ha recibido una nueva versión de la vista Razor.</p>
</section>
PASO 1.1.5 (fin) *@
```

En Razor, `@*` abre un comentario de servidor y `*@` lo cierra. **Mientras los delimitadores sigan presentes, el bloque HTML no llegará al navegador**.

Eliminar completamente la línea `@* PASO 1.1.5 (inicio)` y la línea `PASO 1.1.5 (fin) *@`, pero **conservar el bloque `<section>` interior**. Debe quedar:

```
<section id="marca-paso-1-1-5">
    <h2>Contenido añadido durante la práctica</h2>
    <p>El navegador ha recibido una nueva versión de la vista Razor.</p>
</section>
```

Guardar, reiniciar `dotnet run` y recargar. En la página, debajo del mensaje inicial, debe verse **Contenido añadido durante la práctica**. En F12 → Red → Response debe aparecer también el elemento `<section id="marca-paso-1-1-5">`.

Qué hemos aprendido: modificar una vista modifica el cuerpo HTML de la respuesta. La URL `/` y la acción `Index()` siguen siendo las mismas; ahora el navegador dispone de más elementos para representar.

Los pasos 1.1.4 y 1.1.5 son **compatibles**. Se pueden mantener activados simultáneamente: el primero añade una cabecera, mientras que el segundo añade contenido.

Paso 1.1.6 — Comprobar una ruta válida alternativa y un 404

Con la aplicación ejecutándose, abrir estas direcciones:

```
http://localhost:5091/
http://localhost:5091/Home/Index
http://localhost:5091/ruta-inexistente
http://localhost:5091/css/site.css
```

La primera y la segunda URL deben mostrar **la misma página**. La segunda expresa explícitamente controlador y acción; la primera utiliza los valores por defecto de la ruta convencional.

La tercera debe responder con **404 Not Found**, porque no tenemos una acción correspondiente a `ruta-inexistente`. Es normal que su apariencia sea distinta de la de la página válida: el servidor ha respondido, pero no dispone de una página para ese destino.

La cuarta debe mostrar el texto CSS, no una vista MVC. Esto confirma que una aplicación ASP.NET Core puede servir distintos tipos de recursos HTTP.

Con una segunda terminal, comprobar el 404 sin depender de cómo lo representa el navegador:

```
curl -i http://localhost:5091/ruta-inexistente
```

Si los códigos de respuesta no coinciden con lo esperado, revisar el puerto, la URL exacta y que se esté ejecutando el proyecto `CursoWeb.Mvc` en lugar de `PlantillaMvc`.

D. Ejecución completa y resultados esperados

Al terminar los cambios, mantener el proceso MVC en ejecución con `dotnet run --no-launch-profile --urls http://localhost:5091`. Al visitar `/` deben coexistir: un código HTTP **200**, la cabecera `X-Curso-Paso: 1.1.4` y el contenido HTML añadido en el paso 1.1.5. La hoja de estilos debe cargarse; la URL desconocida debe responder 404.

Este es el comportamiento observable que prueba que los dos fragmentos están activados y que el recorrido principal funciona.

E. Comprobación funcional del alumno

Caso que se debe probar	Operación	Resultado correcto
Plantilla oficial	Abrir el sitio temporal en 5095 tras `dotnet new mvc`	Página de bienvenida MVC
Aplicación del curso	GET `/` en 5091	200 y encabezado `Primer recorrido MVC`
Cabecera del controlador	GET `/` tras activar 1.1.4	`X-Curso-Paso: 1.1.4`
Contenido de la vista	GET `/` tras activar 1.1.5	Visible `Contenido añadido durante la práctica`
Ruta alternativa	GET `/Home/Index`	200 y mismo contenido
Ruta inexistente	GET `/ruta-inexistente`	404
Recurso estático	GET `/css/site.css`	200 y contenido CSS

La prueba automatizada del proyecto comprueba los recorridos HTTP de la aplicación del curso, incluidos los estados base y activado. Se puede ejecutar desde la raíz del repositorio con `bash M01_MVC/TESTS/1.1_e2e.sh` si se dispone de Bash, curl, Python 3 y SDK 8. La creación de la plantilla temporal es una operación didáctica guiada y no modifica la aplicación distribuida.

Criterio de finalización: el alumno debe poder señalar, sobre una petición real, dónde están el método GET, el código 200/404, el encabezado de respuesta, la acción MVC y el HTML de la vista.

F. Problemas frecuentes y soluciones

La terminal indica que `dotnet` no existe. El SDK no está instalado o no está disponible en el PATH. Instalar o reparar SDK .NET 8 y comprobar `dotnet --list-sdks` antes de continuar.

`dotnet new mvc -f net8.0` no reconoce el framework. Comprobar con `dotnet --info` que el SDK 8 está instalado y que no existe un `global.json` que obligue a otro SDK incompatible. No cambiar a net9.0 o net10.0 para continuar el curso.

El puerto 5091 está ocupado. Cerrar otra instancia de la misma aplicación o identificar qué proceso usa el puerto. Es posible seleccionar temporalmente otro puerto mediante `--urls`, pero en ese caso habrá que actualizar todas las URL de la práctica.

Se abre la plantilla en lugar de la página del curso. Se está ejecutando `PlantillaMvc` en el puerto 5095 o se abrió otro proyecto. Detener esa aplicación y arrancar desde `M01_MVC/PROYECTO/1.1/CursoWeb.Mvc` en 5091.

El navegador muestra una pantalla de error de conexión. A diferencia de un HTTP 404, eso suele significar que no existe un servidor escuchando en ese puerto o que se cerró el proceso. Comprobar la consola de `dotnet run`.

No aparece la cabecera personalizada. Comprobar que se quitó el comentario de la línea correcta, que el archivo se guardó, que el proceso se reconstruyó y que se está inspeccionando la respuesta de `/`, no la de `site.css`.

No aparece el contenido adicional en la vista. Revisar que se quitaron *ambos* delimitadores de comentario Razor, que se mantuvo intacto el elemento `<section>` y que se recargó el documento. Para descartar una copia anterior usar una actualización completa en el navegador.

Se recibe un 404 en `/Home/Index`. Verificar que existen `HomeController` e `Index()`, el patrón de `MapControllerRoute` y que se ejecuta la aplicación del curso. En 1.4 veremos cómo diagnosticar errores de rutas con mayor detalle.

En Windows, `curl` parece devolver un objeto en vez de texto HTTP. Dependiendo de la versión de PowerShell, `curl` puede ser un alias de otro comando. Ejecutar `curl.exe -i http://localhost:5091/` o usar la pestaña Red del navegador.

G. Consolidación y reto 1.1 — distinguir metadatos HTTP y HTML

G.1. Observación y razonamiento

Con la aplicación funcionando, verifica que `/` y `/Home/Index` corresponden a la misma acción; compara el documento HTML con `/css/site.css`; explica por qué una URL inexistente recibe 404 sin que falle el servidor. Identifica qué cambia cuando activas el paso 1.1.4 (cabecera) y el 1.1.5 (cuerpo HTML).

G.2. Reto práctico — agregar una segunda cabecera y un texto de diagnóstico

Objetivo: modificar simultáneamente los metadatos de respuesta y el contenido de una vista, demostrando que pertenecen a responsabilidades diferentes. El código del reto **ya está en el proyecto**, pero permanece comentado para que puedas activarlo. Primero resuelve el reto y después compara con el código proporcionado.

Preparación: utiliza `M01_MVC/PROYECTO/1.1/CursoWeb.Mvc` y deja activados los pasos 1.1.4 y 1.1.5. Detén la aplicación antes de editar.

Acción 1 — controlador. En `Controllers/HomeController.cs`, localiza las líneas marcadas `RETO 1.1`. Descomenta exclusivamente la asignación:

```
// RETO 1.1 — Cambiar una cabecera de respuesta sin tocar la vista.
Response.Headers["X-Curso-Reto"] = "HTTP-y-HTML";
```

Esta instrucción añade la cabecera `X-Curso-Reto` a la respuesta HTTP del documento principal, sin modificar el HTML y sin afectar a la respuesta del CSS. Almacena la cadena «HTTP-y-HTML» como valor de la cabecera, no como texto de pantalla.

Acción 2 — vista. En `Views/Home/Index.cshtml`, busca el bloque `RETO 1.1 (inicio)`. Elimina **solo** las dos líneas que delimitan el comentario Razor `@* ... *@`, dejando este HTML activo:

```
<p id="reto-1-1">El servidor genera el HTML y envía las cabeceras HTTP por separado.</p>
```

El atributo `id` facilita localizar el nuevo elemento en las herramientas de desarrollo y en las pruebas automatizadas. El texto se enviará dentro del cuerpo HTML y aparecerá bajo el contenido existente. La vista no está añadiendo por sí misma ninguna cabecera HTTP.

Acción 3 — ejecutar. Desde la carpeta del proyecto:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Accede a `http://localhost:5091/`. Debes ver el texto «El servidor genera el HTML y envía las cabeceras HTTP por separado». En F12 → Red → solicitud del documento → cabeceras de respuesta, comprueba `X-Curso-Reto: HTTP-y-HTML`. También puedes ejecutar `curl -i http://localhost:5091/` desde otra terminal.

Acción 4 — contrastar. Visita `/css/site.css` y verifica que no aparece la cabecera personalizada; responde una solicitud atendida por archivos estáticos, no por `HomeController.Index()`.

Código de solución y activación: las instrucciones anteriores contienen la solución completa para comparar con tu intento. No hay que crear nuevos archivos ni copiar código externo. El reto se activa **sobre el proyecto que ya contiene todos los bloques identificados**.

Prueba automatizada del reto: desde la raíz del repositorio, `bash M01_MVC/TESTS/1.1_e2e.sh` incluye el estado `reto`. Ejecuta en una copia temporal la aplicación con los ejercicios anteriores y el reto activados; verifica que compila, que devuelve HTTP 200, que existe `X-Curso-Reto` y que el párrafo aparece únicamente al activar la variante.

Resultado esperado: distinguir, señalando el archivo y la línea concreta, un cambio de cabeceras (controlador) de un cambio de representación (vista). El reto tiene una solución guiada y comprobable; no implica nuevos conceptos fuera de 1.1.

H. Continuidad del proyecto

Conservar el proyecto CursoWeb.Mvc y los archivos modificados. En los siguientes capítulos añadiremos modelos de productos, nuevas acciones y vistas sin repetir toda la creación inicial. No introducimos EF Core todavía: aparecerá en el día 2. La aplicación temporal `.work/PlantillaMvc` se puede eliminar una vez comprendida la plantilla.

P1.2. El flujo Modelo–Vista–Controlador

A. Identificación

Objetivo. Comprender la función de los tres elementos del patrón MVC, ejecutando el recorrido completo de una petición hacia un controlador de productos, entregando una colección de modelos a la vista y activando progresivamente su presentación y navegación.

Teoría de referencia. Capítulo 1.2 de `M01_TEORIA.md`: responsabilidades, comunicación entre piezas, `View(productos)`, vistas tipadas y separación de datos frente a marcado.

Conocimientos previos. Se presupone completado el punto 1.1 y conocimiento de listas y clases sencillas de C#. No se necesita EF Core ni ninguna base de datos.

Resultado concreto. Una página `/Productos` que presenta la cantidad de productos y tres registros con su nombre y precio, enlazada desde la cabecera del sitio. Los datos se mantienen exclusivamente en memoria.

Duración de referencia. El plan docente reserva 25 minutos de explicación y unos 35 minutos de trabajo práctico. Se ofrece material de lectura y comprobación detallado que también puede utilizarse fuera del aula; dentro de la sesión, el alumnado realizará los cambios indicados y verificará los resultados más relevantes.

B. Preparación y punto de partida

El material se construye como **segunda evolución del mismo proyecto MVC**; para que pueda abrirse y probarse sin depender de copias manuales, la versión del capítulo tiene una carpeta propia:

```
M01_MVC/
  PROYECTO/
    1.1/
      CursoWeb.Mvc/           # Estado conservado del capítulo anterior
    1.2/
      CursoWeb.Mvc/           # Continuación del mismo proyecto, con material del punto 1.2
        CursoWeb.Mvc.csproj
        Program.cs
        Models/
          Producto.cs
        Controllers/
          HomeController.cs
          ProductosController.cs
        Views/
          Home/
            Index.cshtml
          Productos/
            Index.cshtml
          Shared/
            _Layout.cshtml
          _ViewStart.cshtml
          _ViewImports.cshtml
        wwwroot/
          css/
            site.css
        ...
```

No editar el proyecto 1.1 para realizar este capítulo. El proyecto 1.2 contiene su funcionalidad anterior y los nuevos archivos necesarios. Cuando terminemos M01, las sucesivas versiones formarán una secuencia didáctica verificable.

Desde la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.2/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/` y comprobar que sigue apareciendo la página de bienvenida del 1.1. Es la primera comprobación de **no regresión**: introducir un controlador nuevo no debe estropear el existente.

Abrir después `http://localhost:5091/Productos`. Debe aparecer:

- Título: **1.2 | Modelo, Vista y Controlador**.
- Un texto que explica que la vista presenta datos.
- Mensaje: **El controlador todavía no ha entregado una lista a la vista**.

En este estado, la dirección es válida y responde 200, pero aún no se ha activado la entrega de una lista al resultado de vista.

Antes de seguir, detener la aplicación con **Ctrl+C**; reiniciaremos el proceso entre estados. El alumno puede dejar la terminal abierta en la carpeta del proyecto.

C. Desarrollo paso a paso

Paso 1.2.1. Seguir la petición hasta el controlador

Abrir `Program.cs` y localizar la llamada:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Cuando la URL es `/Productos`, el sistema de rutas interpreta que el segmento `Productos` representa el controlador, mientras que `Index` es la acción predeterminada. El método situado en `Controllers/ProductosController.cs` es:

```
[HttpGet]
public IActionResult Index()
{
    // Dentro encontraremos los bloques de código del ejercicio.
    return View();
}
```

Este fragmento muestra solamente las instrucciones activas que explican el comportamiento inicial. La versión real contiene, además, el bloque comentado que habilitaremos en un paso posterior.

Comprueba la relación con la vista abriendo `Views/Productos/Index.cshtml`. Es la ruta convencional que utiliza `return View();` desde la acción `Index` del controlador `ProductosController`. La vista muestra un mensaje porque todavía no recibe ningún dato.

Anota estas tres correspondencias:

Elemento	Ruta o tipo del proyecto	Responsabilidad
URL	`/Productos`	Petición del navegador
Controlador	`Controllers/ProductosController.cs`	Atiende la acción `Index`

Elemento	Ruta o tipo del proyecto	Responsabilidad
Vista	`Views/Productos/Index.cshtml`	Genera la presentación HTML

Comprobación: el navegador debe mostrar una página de `Productos` aunque la lista esté vacía por ausencia de modelo. Un 200 con mensaje no es un 404.

Paso 1.2.2. Reconocer la clase que representa un producto

Abrir `Models/Producto.cs` y examinar la clase que se ha suministrado:

```
namespace CursoWeb.Mvc.Models;

public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
}
```

Qué debes identificar: `Id` distingue un producto de otro; `Nombre` recoge el texto que verá el usuario; `Precio` almacena una cantidad decimal. La clase no importa `Microsoft.AspNetCore.Mvc`, no usa `HttpContext` y no conoce archivos `.cshtml`.

No es necesario descomentar nada en este archivo: es un tipo C# válido y **ya está activo**. El objetivo del paso es reconocer el contrato de datos con el que trabajarán el controlador y la vista. La primera página del proyecto puede continuar funcionando aunque esta clase exista y aún no se utilice.

Pregunta de comprobación: ¿qué propiedad tendrías que consultar para mostrar el nombre de un producto? ¿Sería necesario acceder al navegador desde la clase? Debes poder responder `Nombre` y «no», respectivamente.

Paso 1.2.3. Activar el código que entrega modelos a la vista

Abrir `Controllers/ProductosController.cs`. Dentro del método `Index` localizar el siguiente bloque, que se proporciona comentado:

```
/* PASO 1.2.3 INICIO
var productos = new List<Producto>
{
    new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
    new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
    new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
};

return View(productos);
PASO 1.2.3 FIN */

return View(); // PASO 1.2.3: comentar al activar el envío de datos.
```

En el archivo real, cada línea está indentada dentro del método. No cambies el contenido ni su posición.

Acción 1: eliminar completamente **las dos líneas** que empiezan por `/* PASO 1.2.3 INICIO` y `PASO 1.2.3 FIN */`. Conservar las líneas que crean la lista y devuelven `View(productos)`.

Acción 2: comentar la **última** instrucción `return View();` que quedaba activa en la acción. No basta con quitar los delimitadores: si dejamos ambas instrucciones de retorno activas, la segunda será inalcanzable y habremos dejado código confuso.

El resultado funcional debe tener esta forma:

```
[HttpGet]
public IActionResult Index()
{
    var productos = new List<Producto>
    {
        new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
        new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
        new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
    };

    return View(productos);

    // return View(); // PASO 1.2.3: comentar al activar el envío de datos.
}
```

Explicación:

1. `new List<Producto>` crea una colección en memoria. Todavía no hay base de datos.
2. Cada inicializador `new()` crea un objeto de tipo `Producto`; C# conoce su tipo por la declaración de la colección.
3. Los valores `4.50m`, `1.20m` y `3.75m` son cantidades decimales.
4. `View(productos)` selecciona la vista y le proporciona esa lista como modelo. El controlador no genera manualmente un documento HTML.

Guardar y comprobar desde la carpeta del proyecto:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Visitar `http://localhost:5091/Productos`. El mensaje sin modelo debe haber desaparecido y debe mostrarse: **El controlador ha enviado 3 productos**. Todavía **no** aparece la lista de nombres. Esto es correcto: el controlador ya ha entregado información, pero el bloque que la representa detalladamente sigue comentado.

Detener el servidor antes de continuar.

Paso 1.2.4. Activar la vista que representa los productos

Abrir `Views/Productos/Index.cshtml`. En la primera línea observarás:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

La directiva `@model` especifica que la vista espera una lista de productos de solo lectura o, en el estado inicial, un modelo `null`. Esto no impide que el controlador construya una `List<Producto>` normal: dicha lista implementa `IReadOnlyList<Producto>`.

Localizar el bloque HTML y Razor del ejercicio:

```
@* PASO 1.2.4 (inicio)
<ul id="listado-productos">
@foreach (var producto in Model)
{
    <li data-id="@producto.Id">@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
}
</ul>
PASO 1.2.4 (fin) *@
```

Acción: eliminar únicamente las dos líneas delimitadoras `@* PASO 1.2.4 (inicio)` y `PASO 1.2.4 (fin) *@`. Mantener intacto el bloque `<ul>` y su `@foreach`.

En la versión real, el bloque está dentro del `else` que se ejecuta cuando `Model` no es `null`. No saques el `@foreach` de ese `else`: mantener la estructura permite al ejemplo distinguir correctamente los dos estados.

Después de guardar y ejecutar:

```
dotnet run --no-launch-profile --urls http://localhost:5091
```

Acceder de nuevo a `/Productos`. Ahora deben aparecer el contador y **los tres nombres**: Cuaderno, Lápiz y Carpeta, cada uno con su precio. Puede haber diferencias de separador decimal según la configuración regional; para esta prueba interesa identificar los productos y su cantidad.

Qué cambió: hemos activado la presentación. No hemos añadido productos al controlador ni cambiado la URL. La vista usa los valores existentes en la propiedad `Model`.

Observación adicional: pulsa F12 → Red/Network, recarga `/Productos` y examina el HTML de respuesta. Busca `id="listado-productos"`. El navegador ha recibido elementos `<li>`; no recibe objetos C# ni ejecuta `@foreach`.

Detener el proceso antes del siguiente paso.

Paso 1.2.5. Activar navegación reutilizando el layout

Abrir `Views/Shared/_Layout.cshtml`, localizar el enlace activo Inicio y, justo después, el fragmento comentado:

```
<a href="/">Inicio</a>
@* PASO 1.2.5 (inicio)
<a href="/Productos">Productos</a>
PASO 1.2.5 (fin) *@
```

Retirar las dos líneas que delimitan el comentario Razor de **1.2.5**, conservando los dos enlaces. No cambiar `@RenderBody()` ni el resto del layout.

Reiniciar la aplicación, abrir `/` y comprobar que la cabecera incluye los enlaces **Inicio** y **Productos**. Pulsar Productos; debe mostrarse la misma lista de tres productos que acabamos de activar. Pulsar Inicio para regresar al contenido de 1.1.

Qué hemos aprendido: cambiar el menú no obliga a tocar la clase `Producto` ni el controlador. El layout es una pieza de presentación reutilizable. En el capítulo 1.5 profundizaremos en layouts y vistas parciales; aquí interesa únicamente la separación entre navegación, selección de datos y representación.

Comprobación extra: escribir `http://localhost:5091/Productos/Index` y confirmar que conduce a la misma página que `/Productos`. El funcionamiento se debe al enrutamiento convencional de `Program.cs`.

Paso 1.2.6. Explicar el recorrido con el código abierto

Con la página Productos cargada, reproduce el recorrido usando los archivos del proyecto:

1. El navegador solicita `GET /Productos`.
2. La ruta convencional conduce a `ProductosController.Index()`.
3. La acción crea tres instancias de `Producto` y las reúne en una colección.
4. `return View(productos)` entrega la colección a la vista correspondiente.

5. La vista comprueba que `Model` no es `null`, lee `Model.Count` y recorre los elementos.
6. El layout rodea el resultado con navegación; el servidor envía el HTML.
7. El navegador interpreta el HTML y muestra la lista.

Resultado esperado: debes poder identificar el archivo que interviene en cada una de estas etapas. No es suficiente decir «lo hace MVC»: hay que diferenciar qué pieza hace qué trabajo.

D. Resultado final y ejecución

Con los tres bloques habilitados, el proyecto compila y `http://localhost:5091/Productos` muestra:

```
Inicio   Productos

1.2 | Modelo, Vista y Controlador
La vista decide cómo presentar los datos, pero no inventa el catálogo.

El controlador ha enviado 3 productos.

• Cuaderno — 4.50 €
• Lápiz — 1.20 €
• Carpeta — 3.75 €
```

Los textos decimales pueden adoptar el separador regional del proceso. El HTML real incluye el layout, estilos y atributos adicionales. Lo importante es comprobar que el recuento y los tres registros proceden de la colección entregada a la vista.

La página inicial `/` continúa funcionando. La ruta `/Productos/Index` es equivalente a `/Productos`. Un recurso desconocido debe seguir respondiendo 404.

E. Comprobación funcional por estados

Estado probado	Cambios activados	Resultado observable
Inicial	Ninguno	`/Productos` devuelve 200 y muestra «todavía no ha entregado una lista»
Entrega de datos	1.2.3	Aparece el contador de 3 productos, pero no la lista
Vista activada	1.2.3 + 1.2.4	Aparecen contador y tres productos; aún no existe el enlace adicional
Navegación activada	1.2.3 + 1.2.4 + 1.2.5	Se añade el enlace Productos y permite navegar al catálogo

Para realizar una comprobación automatizada, desde la **raíz del repositorio**:

```
bash M01_MVC/TESTS/1.2_e2e.sh
```

Este script construye el proyecto en su estado inicial y activa las demás combinaciones mediante copias temporales. Arranca realmente Kestrel y verifica las respuestas HTTP, los textos visibles en el HTML, las rutas, los recursos estáticos y la permanencia de la página de 1.1. Requiere SDK .NET 8, Bash, Python 3 y curl; se utiliza también en GitHub Actions. No sustituye la observación de la interfaz con un navegador durante la clase.

Criterio de finalización del ejercicio: puedes explicar por qué el listado no aparece en el estado inicial, por qué el contador aparece antes de activar `@foreach` y por qué el enlace adicional no requiere cambiar el controlador.

F. Problemas previsibles y soluciones

Error de compilación por código inalcanzable o varias instrucciones de retorno. En 1.2.3 se olvidó comentar la instrucción `return View();` original. Debe quedar solamente una ruta de retorno activa en la acción, la que ejecuta `View(productos)`.

La página sigue diciendo que no recibió modelo. Comprobar que los delimitadores `/* ... */` del paso 1.2.3 se quitaron, que la instrucción `return View(productos);` está activa y que se ha reiniciado la aplicación.

El contador muestra tres, pero no hay lista. Todavía no se ha activado el bloque `@foreach` de 1.2.4. Es un estado didáctico válido, no un error del controlador.

Se obtiene un error Razor al activar la lista. Comprobar que se quitaron ambas líneas delimitadoras `@* ... *@` y que permanecen bien emparejadas las llaves del `foreach` y las etiquetas `<ul>` y `<li>`.

Aparece el listado pero no el enlace Productos. El menú depende de `Views/Shared/_Layout.cshtml`, no de la vista `Views/Productos/Index.cshtml`. Activar el bloque 1.2.5 y recargar desde la página inicial.

El navegador abre un proyecto anterior. La carpeta del capítulo 1.1 tiene el mismo nombre de proyecto, porque es una versión conservada; hay que arrancar explícitamente `PROYECTO/1.2/CursoWeb.Mvc`. No ejecutar ambas copias sobre el mismo puerto.

Las cantidades monetarias usan otro separador. `ToString("0.00")` utiliza la configuración regional del proceso cuando no se especifica una cultura. La finalidad de este ejercicio es recorrer datos del modelo, no configurar internacionalización.

Cambiar la clase `Producto` no modifica mágicamente los valores ya enviados. La clase define qué atributos existen; son los objetos creados en la acción los que contienen los valores concretos del catálogo. Modificar propiedades de un tipo y modificar las instancias son operaciones diferentes.

Un 404 en `/Productos`. Comprobar que `ProductosController.cs` está en `Controllers`, que existe `Index()` y que el programa registra la ruta convencional. No confundir esta URL con `/api/productos`, que no existe en nuestra aplicación MVC.

G. Consolidación y reto 1.2 — modificar los datos sin reescribir la vista

G.1. Enunciado

Con todos los pasos 1.2.3, 1.2.4 y 1.2.5 activos y la página `/Productos` funcionando, cambia el primer producto de **Cuaderno** a **Bloc**, manteniendo `Id = 1`, `Precio = 4.50m` y la cantidad total de **tres** productos. No toques `Views/Productos/Index.cshtml`.

Pregunta que debes resolver: si cambia el nombre en el controlador pero no se modifica el bucle `@foreach` de la vista, ¿por qué cambia automáticamente el texto mostrado al usuario?

G.2. Código preparado dentro del proyecto

Abre `M01_MVC/PROYECTO/1.2/CursoWeb.Mvc/Controllers/ProductosController.cs`. Dentro de la lista del **paso 1.2.3**, una vez retirados los delimitadores de comentario de dicho paso, encontrarás las siguientes líneas:

```
new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
// RETO 1.2 — Comentar la línea de Cuaderno y descomentar la siguiente:
// new() { Id = 1, Nombre = "Bloc", Precio = 4.50m },
```

```
new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
```

Acción 1: comenta únicamente la primera declaración de producto (Cuaderno), añadiendo `//` delante.
Acción 2: elimina las dos barras `//` delante de la declaración de Bloc. No modifiques las restantes.

Así queda la parte relevante de la solución, con la variante anterior conservada como referencia:

```
// new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
// RETO 1.2 — Comentar la línea de Cuaderno y descomentar la siguiente:
new() { Id = 1, Nombre = "Bloc", Precio = 4.50m },
new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
```

Por qué funciona: la lista `productos` sigue conteniendo tres instancias de `Producto`. La primera tiene un valor diferente para `Nombre`; la vista simplemente accede a `producto.Nombre` en su bucle y, por eso, mostrará «Bloc». No hay lógica de negocio nueva ni se ha duplicado la definición de la vista.

G.3. Ejecutar y comprobar el reto

Desde el directorio `M01_MVC/PROYECTO/1.2/CursoWeb.Mvc`:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/Productos`. Debes obtener **Bloc, Lápiz y Carpeta**, conservando los precios y el contador de tres productos. Comprueba que ya no aparece «Cuaderno» en la lista. La ruta alternativa `/Productos/Index` debe mostrar exactamente lo mismo.

Prueba automatizada del reto: desde la raíz del repositorio, `bash M01_MVC/TESTS/1.2_e2e.sh`. El escenario `reto` reconstruye una copia temporal del proyecto con el listado y la navegación activados, cambia la línea de Cuaderno por la de Bloc, compila y verifica realmente la respuesta HTTP y sus tres productos. El estado inicial del alumno no se modifica.

G.4. Preguntas y solución razonada

¿Qué archivo cambiarías para mostrar los nombres en una etiqueta HTML diferente? `Views/Productos/Index.cshtml`, ya que decide la presentación.

¿Qué cambiarías para modificar el precio inicial? El valor de `Precio` del objeto en la acción `ProductosController.Index()`; la vista lo consume con `producto.Precio`.

¿Por qué el cambio se pierde si se revierte el código o se reinicia sin recompilarlo? Los datos del catálogo son creados en memoria por el método; no son registros de una base de datos. La persistencia se introducirá en el día 2.

Conclusión del reto: se han utilizado **dos variantes de código ya presentes** en el proyecto; se ha ejecutado el nuevo estado sin inventar piezas ausentes y se ha demostrado la separación entre suministro de datos y representación.

H. Continuidad hacia 1.3

Conservar el resultado de este capítulo. En `1.3` estudiaremos detalladamente las carpetas y convenciones que acabamos de recorrer y prepararemos la organización del proyecto para ampliar más adelante las acciones y las vistas.

No añadir ahora formularios, EF Core, autenticación ni servicios externos. En este capítulo se ha demostrado una arquitectura MVC mínima, suficiente para visualizar y comprender el paso de objetos del controlador a la vista.

P1.3. Estructura y convenciones de un proyecto ASP.NET Core MVC

A. Identificación

Objetivo práctico: identificar dónde se encuentra cada parte de una aplicación MVC, comprender la diferencia entre arranque y atención de peticiones, leer un valor de configuración, añadir recursos estáticos y comprobar qué rutas son públicas mediante `UseStaticFiles()`. El reto final introduce una segunda URL de texto accesible, cuyo enlace se activará utilizando el **código que ya se encuentra en el proyecto**.

Teoría asociada: apartado 1.3 completo de `M01_TEORIA.md`. Se utilizarán los conceptos ya estudiados de HTTP y MVC de 1.1 y 1.2.

Estado de partida: proyecto del capítulo **1.3**, que conserva y deja activos los pasos funcionales 1.2.3, 1.2.4 y 1.2.5. La página de productos puede abrirse desde el principio con tres registros. No es necesario repetir el desbloqueo del controlador y la vista del capítulo anterior.

Duración de aula prevista: 20 minutos de explicación y 35 minutos de práctica; la extensión de ejemplos y comprobaciones del documento permite repasar o completar observaciones fuera de ese tiempo. Las comprobaciones orientadas a docentes se automatizan para no consumir tiempo lectivo de los alumnos.

B. Preparación del laboratorio

Necesitamos **SDK .NET 8**, Visual Studio con soporte ASP.NET Core o un editor compatible con C#, navegador web con herramientas de desarrollo y terminal. En este punto no se usa base de datos, API externa ni Docker.

Desde una copia del repositorio y posicionándose **en su raíz**:

```
cd M01_MVC/PROYECTO/1.3/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/`. Debe aparecer la portada ya conocida. Pulsar `Productos` en la navegación. La dirección `http://localhost:5091/Productos` debe mostrar **Cuaderno, Lápiz y Carpeta**, con el contador de tres productos.

Observación inicial: el enlace a productos y la vista de productos están activos porque son la base funcional de 1.3. No es una aplicación nueva; es la continuación del mismo ejemplo didáctico, guardada en una carpeta autónoma para que todas las etapas permanezcan disponibles.

Detener el servidor con **Ctrl+C** y mantener abierta la carpeta del proyecto. Los cambios de código se probarán reiniciándolo para identificar sin ambigüedad el estado ejecutado.

C. Desarrollo paso a paso

Paso 1.3.1 — Inspeccionar la estructura real del proyecto

Desde el explorador de soluciones de Visual Studio o desde el explorador de archivos, identificar:

```
M01_MVC/PROYECTO/1.3/CursoWeb.Mvc/
├── CursoWeb.Mvc.csproj
├── Program.cs
├── appsettings.json
├── Properties/launchSettings.json
├── Controllers/HomeController.cs
├── Controllers/ProductosController.cs
├── Models/Producto.cs
├── Views/Home/Index.cshtml
├── Views/Productos/Index.cshtml
├── Views/Shared/_Layout.cshtml
├── Views/_ViewStart.cshtml
├── Views/_ViewImports.cshtml
├── wwwroot/css/site.css
├── wwwroot/css/catalogo.css
├── wwwroot/recursos/guia-publica.txt
├── wwwroot/recursos/aviso-laboratorio.txt
└── DOCUMENTACION/nota-interna.txt
```

No modificar todavía ningún archivo. El objetivo es asociar cada necesidad a una ubicación razonable.

Completar verbalmente las siguientes relaciones:

Necesidad	Archivo o carpeta donde buscar
Cambiar el tipo de proyecto o framework	`CursoWeb.Mvc.csproj`
Comprender qué se registra al arrancar	`Program.cs`
Mostrar otro producto en la lista fija	`Controllers/ProductosController.cs`
Modificar el marcado HTML del listado	`Views/Productos/Index.cshtml`
Encontrar las propiedades de un producto	`Models/Producto.cs`
Añadir un enlace común a todas las páginas	`Views/Shared/_Layout.cshtml`
Añadir una hoja CSS pública	`wwwroot/css/`
Definir ajustes no sensibles del curso	`appsettings.json`

Por qué esta identificación es útil: al ampliar una aplicación, la mayor parte de los errores de mantenimiento comienzan por cambiar un archivo que no es responsable del comportamiento. Saber dónde mirar reduce modificaciones innecesarias y favorece la separación MVC.

Comprueba además que `bin/` y `obj/`, cuando aparecen tras compilar, son resultados generados; no forman parte del código del alumno que debe versionarse.

Paso 1.3.2 — Entender Program.cs antes de tocar una página

Abrir `Program.cs` e identificar el siguiente recorrido:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Se omite aquí únicamente el bloque opcional del paso 1.3.3.
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
```

```
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

El fragmento representa las instrucciones activas y esenciales. El archivo completo contiene también comentarios docentes.

Explicación de cada instrucción:

1. `CreateBuilder(args)` prepara la aplicación y las fuentes de configuración.
2. `AddControllersWithViews()` registra los servicios de MVC.
3. `Build()` construye el host.
4. `UseStaticFiles()` permite enviar CSS y texto desde `wwwroot` cuando sus tipos son reconocidos.
5. `UseRouting()` participa en el enrutamiento.
6. `MapControllerRoute` registra una convención que permite llegar a `HomeController.Index` y `ProductosController.Index`.
7. `Run()` arranca el servidor. La acción de productos se ejecutará cuando llegue una petición, no por el mero hecho de registrar la ruta.

Comprobación: sin añadir ningún código, abre `/Productos` y `/css/site.css` y observa que ambos devuelven HTTP 200, pero el primero produce HTML MVC y el segundo CSS. Se atienden solicitudes de naturaleza distinta dentro de una misma aplicación ASP.NET Core.

Paso 1.3.3 — Leer un valor desde appsettings.json

Primero abrir `appsettings.json` y localizar:

```
"Curso": {
  "Nombre": "Curso MVC de productos",
  "Edicion": "Laboratorio NET8"
}
```

Este fragmento forma parte de un JSON completo que ya se encuentra correctamente configurado. No se deben crear comentarios de estilo C# dentro de `appsettings.json`, porque el ejercicio no los necesita y podrían volver inválido el archivo.

Acción: abrir `Program.cs` y localizar la instrucción comentada con la marca `PASO 1.3.3`:

```
// app.Logger.LogInformation("P1.3 configuracion: {Nombre}", app.Configuration["Curso:Nombre"]);
```

Quitar **solo** las dos barras `//` delante de la instrucción. No eliminar el resto de la configuración. La línea activa queda:

```
app.Logger.LogInformation("P1.3 configuracion: {Nombre}", app.Configuration["Curso:Nombre"]);
```

Cómo funciona: `app.Configuration["Curso:Nombre"]` obtiene el valor situado en la sección `Curso`, clave `Nombre`, cargada desde la configuración. `app.Logger.LogInformation` emite un mensaje de nivel *Information* en el registro del proceso; `{Nombre}` es un marcador de valor del sistema de logging.

Construir y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

En la consola de ejecución debe aparecer una entrada equivalente a:

```
P1.3 configuracion: Curso MVC de productos
```

La estructura y los prefijos de fecha o categoría del registro pueden variar; el contenido es lo relevante.

Resultado esperado: la aplicación sigue mostrando sus dos páginas anteriores; la instrucción nueva escribe **en el registro de la aplicación**, no en el HTML. Se demuestra así que un ajuste de `appsettings.json` puede leerse durante el arranque sin tocar una vista.

Detener el proceso antes del siguiente paso.

Paso 1.3.4 — Incorporar una hoja CSS sin crear una acción MVC

La carpeta `wwwroot/css` incluye `catalogo.css`, cuyo contenido ya está activo:

```css
/* PASO 1.3.4: recurso estático independiente de los controladores. */
/* La regla está activa en este archivo; el alumno activa su enlace en _Layout.cshtml. */
main.container {
    border-left: 5px solid #2d635d;
    padding-left: 1.5rem;
}
```

Qué hace: cambia la presentación del elemento HTML `<main class="container">`, añadiendo un borde izquierdo y espacio interior. No contiene Razor ni código C#, y por ello el controlador no necesita modificarse.

Acción: abrir `Views/Shared/_Layout.cshtml`. Bajo la referencia a `site.css`, localizar:

```
@* PASO 1.3.4 (inicio)
<link rel="stylesheet" href="/css/catalogo.css" />
PASO 1.3.4 (fin) *@
```

Eliminar las **dos líneas** que delimitan ese comentario Razor. Dejar activa exactamente la siguiente etiqueta, manteniendo la hoja anterior:

```
<link rel="stylesheet" href="~/css/site.css" />
<link rel="stylesheet" href="/css/catalogo.css" />
```

Por qué es necesario: el archivo `catalogo.css` ya existe físicamente en `wwwroot` y puede descargarse si se conoce su URL, pero el navegador solo aplicará sus reglas a la página cuando el HTML recibido incluya una etiqueta `<link rel="stylesheet">` adecuada.

Reiniciar la aplicación y abrir `http://localhost:5091/Productos`. Debe apreciarse un **borde vertical** en el área principal. En F12 → Red, comprobar que además de la hoja `site.css` el navegador solicita `/css/catalogo.css` y recibe HTTP 200 y tipo `text/css`.

Prueba independiente: abre `http://localhost:5091/css/catalogo.css` directamente. Se verá el contenido CSS como respuesta HTTP. Esa URL no pasa por un controlador MVC.

Detener el servidor antes de seguir.

Paso 1.3.5 — Enlazar un recurso de texto publicado desde wwwroot

Dentro de `wwwroot/recursos/` existe el archivo `guia-publica.txt`, cuyo contenido es:

```
PASO 1.3.5 - Guia publica del curso ASP.NET Core MVC.
Este archivo se sirve desde wwwroot mediante UseStaticFiles.
No es necesario crear un controlador para atender esta URL.
```

Acción: abrir `Views/Shared/_Layout.cshtml` y localizar la marca `PASO 1.3.5`, junto a la navegación:

```
@* PASO 1.3.5 (inicio)
<a href="/recursos/guia-publica.txt">Guía pública</a>
PASO 1.3.5 (fin) *@
```

Retirar únicamente los dos delimitadores del comentario Razor. El enlace queda activo:

```
<a href="/recursos/guia-publica.txt">Guía pública</a>
```

Reiniciar la aplicación, abrir `/Productos` y comprobar que ahora hay un enlace **Guía pública** en la navegación. Pulsarlo: el navegador debe pedir `/recursos/guia-publica.txt` y mostrar el texto del archivo, no una vista Razor. Revisar en Red que la solicitud obtiene HTTP 200.

Observación decisiva: prueba también la URL directa `/recursos/guia-publica.txt` antes de descomentar el enlace, si deseas comparar estados. El recurso ya era accesible. Habilitar el enlace facilita encontrarlo en la interfaz, pero **no cambia la configuración de publicación estática**.

Aquí no hemos creado ninguna clase nueva ni un método de controlador. El trabajo ya lo hace el middleware de archivos estáticos.

D. Tabla de ejecución y resultados

Estado	Qué has activado	Verificación
Base funcional de 1.3	Ninguno	`/` y `/Productos` responden 200; se muestran tres productos
Configuración	1.3.3	El registro de inicio muestra `Curso MVC de productos`
Estilo adicional	1.3.3 + 1.3.4	`/Productos` incluye el enlace CSS adicional; el navegador carga el nuevo estilo
Guía pública	1.3.3 + 1.3.4 + 1.3.5	La navegación incluye Guía pública; el texto se descarga con 200
Reto	Todo lo anterior + RETO 1.3	Aparece Aviso de laboratorio; recurso de `wwwroot` 200 y archivo interno 404

Comprobación de continuidad: la portada y el catálogo del capítulo 1.2 deben seguir funcionando en todos los estados; no se ha desactivado ninguno de esos recorridos.

E. Reto 1.3 — Recurso público frente a archivo interno

Enunciado

Se pide añadir un enlace visible llamado **Aviso de laboratorio**, que permita al alumno abrir un archivo de texto público. También debes localizar un segundo archivo de texto que **no se publique** por la configuración actual de `UseStaticFiles()` y explicar por qué produce otra respuesta.

El reto comprueba si sabes razonar a partir de la **estructura física del proyecto**, la raíz web y el layout, y no únicamente repetir los comandos del paso 1.3.5.

Código incluido en el proyecto

No necesitas crear archivos que falten; ya están incorporados en `PROYECTO/1.3/CursoWeb.Mvc`.

Archivo 1 — `wwwroot/recursos/aviso-laboratorio.txt`:

```
RETO 1.3 - Aviso de laboratorio.
El documento esta dentro de wwwroot y puede solicitarse como recurso estatico.
El enlace de navegacion del reto se activa por separado en _Layout.cshtml.
```

Archivo 2 — `DOCUMENTACION/nota-interna.txt`:

```
Nota didactica interna del proyecto (datos ficticios).
Al estar fuera de wwwroot, UseStaticFiles no la publica de forma predeterminada.
No colocar secretos reales en el repositorio: esta nota solo ilustra una convencion.
```

Archivo 3 — `Views/Shared/_Layout.cshtml`, con el siguiente bloque dentro del menú:

```
@* RETO 1.3 (inicio)
<a href="/recursos/aviso-laboratorio.txt">Aviso de laboratorio</a>
RETO 1.3 (fin) *@
```

Resolver y activar el reto

Acción 1: abre `Views/Shared/_Layout.cshtml`, localiza `RETO 1.3 (inicio)` y elimina **las dos líneas que delimitan el comentario**, sin cambiar el contenido del enlace. La solución activa queda:

```
<a href="/recursos/aviso-laboratorio.txt">Aviso de laboratorio</a>
```

Esta etiqueta HTML se añadirá al layout de ambas páginas. No se precisa modificar `ProductosController` ni `Program.cs`: el recurso ya está dentro de `wwwroot` y `UseStaticFiles()` ya forma parte del arranque.

Acción 2: compila y ejecuta de nuevo la aplicación:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/Productos` y pulsa el enlace Aviso de laboratorio. Debe mostrarse el contenido de `aviso-laboratorio.txt`. En la pestaña Red, comprobar el código **200**.

Acción 3: escribe directamente estas direcciones en el navegador o ejecútalas en una segunda terminal con `curl -i`:

```
http://localhost:5091/recursos/aviso-laboratorio.txt
http://localhost:5091/DOCUMENTACION/nota-interna.txt
```

La primera respuesta debe ser HTTP **200** y devolver el contenido del archivo público. La segunda debe devolver HTTP **404**, pues nuestro middleware no sirve por defecto archivos del directorio `DOCUMENTACION`.

Acción 4: explica por escrito por qué el enlace del reto no es un mecanismo de seguridad. La respuesta correcta es que un recurso bajo `wwwroot` **ya es accesible directamente** aunque no aparezca en la navegación. La ausencia del enlace no protege el contenido. Además, aunque `nota-interna.txt` no se sirva por el servidor web, forma parte de un repositorio público: este ejemplo no sirve para guardar secretos.

Solución explicada

El enlace del archivo de layout es la **única línea de HTML** que debe activarse para resolver el reto. Al hacerlo se modifica la navegación, pero la aplicación no necesita un endpoint controlador adicional porque ya conoce cómo servir archivos estáticos.

No cambiamos `wwwroot/recursos/aviso-laboratorio.txt`: el texto estaba disponible desde el inicio. Tampoco movemos `DOCUMENTACION/nota-interna.txt`, porque la comparación de ubicaciones es el objetivo didáctico.

La combinación de resultados confirma qué servicio está atendiendo cada petición:

```
GET /Productos                              -> 200, HTML MVC
GET /recursos/aviso-laboratorio.txt         -> 200, texto público
GET /DOCUMENTACION/nota-interna.txt        -> 404, no publicado como estático
```

Reto entregado con código: el Markdown describe la solución completa; el proyecto contiene los dos textos y el bloque del layout comentado; las pruebas automatizadas habilitan el fragmento y comprueban los tres resultados.

F. Verificación automatizada del ejercicio y el reto

Desde la raíz del repositorio y con la aplicación manual **detenida** para dejar libre el puerto:

```
bash M01_MVC/TESTS/1.3_e2e.sh
```

La comprobación crea copias temporales, activa de manera controlada los bloques de cada etapa y comprueba con peticiones HTTP reales:

- Que las páginas de 1.1 y 1.2 continúan devolviendo HTML y muestran los productos.
- Que el registro de configuración aparece al activar el paso 1.3.3.
- Que el CSS es accesible y solo aparece enlazado en el documento cuando activamos 1.3.4.
- Que la Guía pública es accesible y se añade al menú en 1.3.5.
- Que el reto añade el enlace de aviso, cuyo archivo responde 200, mientras que el archivo exterior a `wwwroot` responde 404.

Es una prueba de integración funcional HTTP usando Kestrel; el navegador real y la comprobación visual del borde se realizan de forma guiada en clase. Una prueba HTTP del enlace y del CSS **no es equivalente a verificar su representación visual automatizada**.

G. Problemas frecuentes y soluciones

No aparecen los tres productos al iniciar el punto 1.3. Comprueba que has abierto `PROYECTO/1.3/CursoWeb.Mvc` y no el proyecto `1.1` o el estado inicial de `1.2`. La versión 1.3 ya incorpora el catálogo funcionando.

Se obtiene una excepción al leer la configuración. Comprueba que no se alteró la sintaxis JSON de `appsettings.json`. La sección `Curso` debe encontrarse dentro de las llaves del objeto raíz y separada mediante comas correctas.

El registro no aparece en la terminal. Verifica que descomentaste la línea de `Program.cs`, que guardaste y reiniciaste el programa y que la configuración incluye `Curso:Nombre`. Comprueba también que no cambiaste el nivel de log para silenciar `Information`.

El CSS devuelve 200, pero no aparece el borde. Comprueba que activaste el `<link>` del paso 1.3.4, que realmente apunta a `/css/catalogo.css`, que recargaste la página y que no hay una regla CSS posterior que invalide el estilo. Consulta la pestaña Red y el inspector de elementos.

El enlace aparece, pero el archivo devuelve 404. Verifica la ruta relativa desde `wwwroot`: la URL no contiene `wwwroot`, sino `/recursos/guia-publica.txt` o `/recursos/aviso-laboratorio.txt`.

Al escribir la URL de `nota-interna.txt` se obtiene 404 y parece un fallo. Es el resultado esperado de la práctica. No hay un archivo dentro de `wwwroot` con esa ruta ni se ha configurado un proveedor alternativo.

Piensas que el archivo es privado porque no hay enlace. Es un error importante. Los enlaces mejoran la navegación; no definen permisos de acceso al recurso. Los archivos de `wwwroot` quedan públicamente disponibles en las condiciones demostradas.

El código de un paso afecta al siguiente. Activa los bloques en orden: configuración → CSS → Guía pública → reto. Si has dejado un comentario Razor sin cerrar, restáuralo o compara exactamente los dos delimitadores con el Markdown.

H. Consolidación y continuidad

Sin modificar el código, responde y justifica sobre los archivos reales:

1. ¿Qué diferencias existen entre `CursoWeb.Mvc.csproj`, `Program.cs` y `appsettings.json`?
2. ¿Por qué `/Productos` devuelve HTML generado por MVC mientras `/css/catalogo.css` devuelve un archivo?
3. ¿Qué hace `_ViewStart.cshtml` y qué hace `_ViewImports.cshtml`?
4. ¿Por qué un archivo en `DOCUMENTACION` no queda servido como estático y por qué no deberíamos guardar secretos en un repositorio público?
5. ¿Qué pasaría si quitáramos el enlace de la Guía pública del layout: el archivo desaparecería de `wwwroot` o seguiría accesible directamente por URL?

Resultado consolidado: localizas las piezas de la aplicación y puedes explicar sus responsabilidades, su ciclo de arranque y la diferencia entre recursos físicos internos y recursos de la raíz web.

Conserva los cambios y continúa en **1.4 — Controladores, acciones, parámetros y routing**, donde ampliaremos el catálogo con una acción de detalle de producto, rutas y resultados HTTP apropiados.

P1.4. Controladores, acciones, parámetros y routing

A. Objetivos, punto de partida y tiempo

Objetivo principal: ampliar el catálogo MVC existente sin rehacerlo. El alumno añadirá acciones a `ProductosController`, observará varios tipos de `IActionResult`, recibirá un identificador desde la URL, construirá una ficha Razor, publicará una ruta por atributos y habilitará enlaces de detalle. El reto propone otra acción por atributos que devuelve JSON, con su código completo tanto en este documento como en el proyecto.

Teoría asociada: capítulo 1.4 del Markdown de teoría, especialmente el patrón de ruta convencional, el paso del parámetro `id`, los resultados `View`, `Content`, `BadRequest` y `NotFound`, y la comparación con `[HttpGet("/...")]`.

Tiempo orientativo de aula: 30 minutos de teoría y 65 minutos de práctica, incluido el reto breve. Los ejemplos completos y las comprobaciones adicionales permiten repasar fuera del horario lectivo sin ampliar el temario obligatorio.

El estado inicial del proyecto de **1.4** contiene una aplicación que ya muestra la lista de productos, con configuración y estilos del punto 1.3 activos. Hemos conservado los datos en memoria. Las acciones de este capítulo y el reto están **comentados**, por lo que el alumno puede habilitarlos por etapas sin añadir código desde cero.

B. Preparación del proyecto y estructura

Ubicación desde la raíz del repositorio:

```
M01_MVC/
├── PROYECTO/
│   ├── 1.1/CursoWeb.Mvc/     (estado anterior conservado)
│   ├── 1.2/CursoWeb.Mvc/     (estado anterior conservado)
│   ├── 1.3/CursoWeb.Mvc/     (estado anterior conservado)
│   └── 1.4/CursoWeb.Mvc/
│       ├── CursoWeb.Mvc.csproj
│       ├── Program.cs
│       ├── Controllers/
│       │   ├── HomeController.cs
│       │   └── ProductosController.cs
│       ├── Models/Producto.cs
│       ├── Views/
│       │   ├── Productos/Index.cshtml
│       │   ├── Productos/Detalle.cshtml
│       │   ├── Home/Index.cshtml
│       │   └── Shared/_Layout.cshtml
│       ├── appsettings.json
│       └── wwwroot/
└── TESTS/1.4_e2e.sh
```

No modificar el proyecto 1.3. El proyecto 1.4 es una continuación autocontenida del estado final funcional anterior. Las carpetas independientes permiten volver a un estado previo si se necesita reproducir la actividad de un capítulo.

Desde una terminal en la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.4/CursoWeb.Mvc
dotnet restore
```

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/Productos`. Debe mostrarse la lista con **Cuaderno, Lápiz y Carpeta**. Abre también `http://localhost:5091/`: debe mantenerse la página de inicio. Este recorrido confirma que el punto de partida no ha perdido la funcionalidad desarrollada.

En este estado aún **no** está activa `/Productos/Detalle/2`: esa URL devuelve 404. Es una situación deliberada que cambiará tras activar el bloque del paso 1.4.3.

Detener el proceso con **Ctrl+C** antes de editar. Volver a iniciar la aplicación después de cada paso. No ejecutar simultáneamente dos versiones `CursoWeb.Mvc` sobre el mismo puerto.

C. Desarrollo guiado

Paso 1.4.1 — Examinar la ruta y preparar datos compartidos

Abrir `Program.cs` y localizar esta ruta, ya activa:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Interpretación: el primer segmento identifica el controlador y, si no se proporciona, se utiliza `Home`. El segundo representa la acción y, si falta, se emplea `Index`. El tercer segmento es un valor de ruta opcional denominado `id`.

Comprueba tres URL:

```
http://localhost:5091/
http://localhost:5091/Productos
http://localhost:5091/Productos/Index
```

La primera muestra la portada; la segunda y tercera muestran el mismo listado, porque las dos llegan a la acción `Index()` de `ProductosController`.

Ahora abrir `Controllers/ProductosController.cs`. Observarás que el catálogo ya no se construye directamente en `Index`, como ocurría en 1.3. Se ha extraído a:

```
private static IReadOnlyList<Producto> ObtenerProductosEjemplo() =>
    new List<Producto>
    {
        new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
        new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
        new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
    };
```

Dentro del archivo existe además una variante histórica comentada del reto 1.2. **No la actives para este capítulo**, porque necesitamos los tres datos de partida tal como se han definido aquí.

La acción `Index` reutiliza el método:

```
[HttpGet]
public IActionResult Index()
{
    return View(ObtenerProductosEjemplo());
}
```

Explicación: el método privado reúne la definición de los datos que queremos consultar desde distintas acciones. No es una URL pública. `Index()` obtiene la colección y devuelve `View(...)`, como ya vimos en 1.2. No se ha añadido una base de datos: cada llamada a ese método construye una colección de ejemplo en memoria.

Comprobación: el contenido visible del listado no ha cambiado. Este es un ejemplo de refactorización limitada que reduce la duplicación sin alterar el resultado observado.

Paso 1.4.2 — Crear una acción que devuelve texto en vez de HTML

Localizar dentro de `ProductosController.cs` el bloque:

```
/* PASO 1.4.2 INICIO
[HttpGet]
public IActionResult Estado()
{
    return Content("Catalogo MVC disponible", "text/plain");
}
PASO 1.4.2 FIN */
```

Acción: eliminar únicamente las líneas de comentario `/* PASO 1.4.2 INICIO` y `PASO 1.4.2 FIN */`. Conservar todos los caracteres del método. Se activará:

```
[HttpGet]
public IActionResult Estado()
{
    return Content("Catalogo MVC disponible", "text/plain");
}
```

Qué hace cada parte: `[HttpGet]` limita el método a solicitudes GET; `Estado()` es el nombre de la acción; `Content(...)` devuelve un cuerpo de texto con tipo `text/plain`. No creamos `Views/Productos/Estado.cshtml` porque no hemos pedido renderizar una vista.

Compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/Productos/Estado`. Se debe mostrar **Catalogo MVC disponible**, sin menú ni layout. En las herramientas de desarrollo del navegador comprobar `GET`, código **200** y tipo de contenido `text/plain`.

Alternativa desde una segunda terminal:

```
curl -i http://localhost:5091/Productos/Estado
```

Razonamiento: `/Productos` y `/Productos/Estado` comparten controlador, pero sus acciones devuelven tipos de resultado diferentes. No hay que asumir que todos los métodos del controlador generan páginas Razor.

Detener el proceso antes de continuar.

Paso 1.4.3 — Activar la ficha de detalle con parámetro id

Primero abrir `Views/Productos/Detalle.cshtml`. La vista ya existe en el proyecto, con su código completo:

```
@model CursoWeb.Mvc.Models.Producto
@{
    ViewData["Title"] = "Detalle del producto";
}
<h1 id="titulo-detalle">1.4 | Detalle de producto</h1>
<dl id="detalle-producto">
    <dt>Identificador</dt><dd id="detalle-id">@Model.Id</dd>
    <dt>Nombre</dt><dd id="detalle-nombre">@Model.Nombre</dd>
    <dt>Precio</dt><dd id="detalle-precio">@Model.Precio.ToString("0.00") €</dd>
</dl>
<p><a href="/Productos">Volver al catálogo</a></p>
```

Explicación de la vista: a diferencia de `Index.cshtml`, que recibía una lista, aquí `@model` declara un único `Producto`. Las expresiones `@Model.Id`, `@Model.Nombre` y `@Model.Precio` generan el contenido de las etiquetas HTML. El enlace final permite regresar al listado sin modificar datos.

Ahora localizar en `ProductosController.cs`:

```
/* PASO 1.4.3 INICIO
[HttpGet]
public IActionResult Detalle(int id)
{
    if (id <= 0)
    {
        return BadRequest("El identificador debe ser mayor que cero.");
    }

    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return View(producto);
}
PASO 1.4.3 FIN */
```

Acción: retirar las dos líneas que delimitan el comentario `PASO 1.4.3`, conservando el bloque interior. Se activará exactamente el método `Detalle(int id)` que se acaba de mostrar.

Explicación de funcionamiento:

1. `Detalle(int id)` recibe el identificador desde el tercer segmento de la ruta convencional.
2. `if (id <= 0)` detecta el caso que el ejercicio considera inválido y devuelve **400 Bad Request**.
3. `ObtenerProductosEjemplo().FirstOrDefault(...)` busca el producto cuyo `Id` coincida.
4. `if (producto is null) return NotFound();` genera **404** cuando no existe el elemento solicitado.
5. `return View(producto);` entrega un único modelo `Producto` a la vista correspondiente y produce HTML si todo es correcto.

Reiniciar el servidor y comprobar estas direcciones:

URL	Resultado correcto
`/Productos/Detalle/2`	200, ficha de **Lápiz** con id 2
`/Productos/Detalle/1`	200, ficha de **Cuaderno**
`/Productos/Detalle/99`	404, identificador inexistente
`/Productos/Detalle/0`	400, identificador rechazado por el controlador

URL	Resultado correcto
`/Productos/Detalle`	400 en este ejemplo, por recibir el valor predeterminado 0

En el navegador, los códigos 400/404 pueden aparecer como páginas de error sencillas. Usar `curl -i` o F12 → Red para comprobar el **código HTTP**, que es la parte evaluada.

Qué distingue 400 de 404: en este ejemplo, 400 corresponde a un identificador que nuestro método rechaza explícitamente; 404 corresponde a un identificador que no está en la colección. Un 404 también puede producirse si ninguna ruta coincide, pero aquí, para `id=99`, el código de la acción sí se ha ejecutado.

Detener el proceso antes del siguiente paso.

Paso 1.4.4 — Añadir una URL por atributos para reutilizar la misma vista

El listado utiliza una ruta convencional como `/Productos/Detalle/2`. Ahora incorporaremos una dirección distinta, `/catalogo/ficha/2`, usando un atributo en el método.

Localizar y activar el bloque `PASO 1.4.4`:

```
/* PASO 1.4.4 INICIO
[HttpGet("/catalogo/ficha/{id:int}")]
public IActionResult Ficha(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return View("Detalle", producto);
}
PASO 1.4.4 FIN */
```

De nuevo, eliminar únicamente las **dos líneas delimitadoras** del comentario. El resto constituye código C# ejecutable.

Aspectos clave:

- `[HttpGet("/catalogo/ficha/{id:int}")]` define la URL completa de la acción y la limita a GET. El segmento `id` tiene una restricción de entero.
- `Ficha(int id)` recibe el mismo tipo de parámetro que `Detalle(int id)`, pero desde una plantilla explícita.
- `return View("Detalle", producto);` **reutiliza** `Views/Productos/Detalle.cshtml`. Es necesario nombrarla porque, sin el nombre, MVC buscaría una vista llamada `Ficha` siguiendo la convención habitual.
- `NotFound()` mantiene el tratamiento del caso en que el identificador es entero pero no corresponde a un producto.

Reiniciar y comprobar:

```
http://localhost:5091/catalogo/ficha/2
http://localhost:5091/catalogo/ficha/99
http://localhost:5091/catalogo/ficha/no-numero
http://localhost:5091/Productos/Ficha/2
```

La primera debe mostrar la ficha de Lápiz con HTTP 200; las otras tres deben responder 404. En el segundo caso, **la acción se ejecuta pero no encuentra el producto**. En el tercero la restricción `{id:int}` impide seleccionar la acción. En el último caso no coincide la URL por atributos definida para `Ficha`: la ruta correcta no es `/Productos/Ficha/2`.

Comparación razonada: las dos acciones `Detalle(2)` y `Ficha(2)` terminan generando el **mismo HTML**, aunque fueron seleccionadas mediante URLs diferentes.

Paso 1.4.5 — Activar enlaces de detalle desde cada producto del listado

Una URL puede funcionar sin que exista aún un enlace visible hacia ella. Ahora añadiremos navegación al listado de forma que cada producto conduzca a su ficha.

Abrir `Views/Productos/Index.cshtml`. Dentro del `@foreach` localizar:

```
<li data-id="@producto.Id">
    @producto.Nombre — @producto.Precio.ToString("0.00") €
    @* PASO 1.4.5 (inicio)
    <a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
    PASO 1.4.5 (fin) *@
</li>
```

Acción: eliminar las dos líneas delimitadoras del comentario Razor `PASO 1.4.5`, conservando el enlace HTML:

```
<li data-id="@producto.Id">
    @producto.Nombre — @producto.Precio.ToString("0.00") €
    <a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
</li>
```

Explicación: `@producto.Id` escribe el identificador del elemento actual en la URL. El mismo bucle produce **tres enlaces**, que apuntan a `/Productos/Detalle/1`, `/Productos/Detalle/2` y `/Productos/Detalle/3`.

Reiniciar la aplicación y abrir `http://localhost:5091/Productos`. Debe aparecer el texto **Ver detalle** junto a cada producto. Pulsa el enlace de Lápiz; comprueba que la ficha muestra id 2, nombre Lápiz y precio. Utiliza el enlace **Volver al catálogo** de la vista de detalle para regresar.

Por qué activamos el enlace al final: antes de crear `Detalle(int id)`, el enlace conduciría a una dirección sin acción válida. Ahora se ha comprobado antes el endpoint y después la navegación.

D. Reto 1.4 — Una acción MVC que devuelve un resultado JSON

Enunciado y objetivo

El controlador MVC produce HTML en `Detalle` y texto plano en `Estado`. El reto consiste en publicar una tercera representación muy sencilla: un **resumen JSON** de un producto, seleccionado por su identificador, mediante una ruta por atributos.

Condiciones:

- URL `GET /catalogo/resumen/{id:int}`.
- Para un producto existente, HTTP 200 y JSON con `id` y `nombre`.
- Para un identificador entero inexistente, HTTP 404.
- Para un segmento no entero, la ruta no debe coincidir y se obtiene HTTP 404.
- No crear una vista `Resumen.cshtml` ni añadir una segunda fuente de datos.

El alumnado debe intentar explicar la solución antes de activarla. El código completo está disponible **comentado en el proyecto** y se reproduce aquí para evitar que el reto dependa de material externo.

Código del reto y explicación

Abrir `Controllers/ProductosController.cs` y localizar:

```
/* RETO 1.4 INICIO
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return Ok(new { producto.Id, producto.Nombre });
}
RETO 1.4 FIN */
```

Acción 1: eliminar las líneas `/* RETO 1.4 INICIO` y `RETO 1.4 FIN */`. Se activa así **exactamente el mismo código**, sin necesidad de crear archivos adicionales.

Cómo funciona cada parte:

1. La plantilla `"/catalogo/resumen/{id:int}"` selecciona solicitudes GET cuyo último segmento puede convertirse a entero.
2. El parámetro `int id` recibe el valor extraído de la URL.
3. `ObtenerProductosEjemplo()` reutiliza el catálogo ya utilizado por `Index`, `Detalle` y `Ficha`.
4. `FirstOrDefault(p => p.Id == id)` busca el registro correspondiente.
5. Si no existe, `NotFound()` genera HTTP 404.
6. `Ok(new { producto.Id, producto.Nombre })` construye una respuesta HTTP 200 con un objeto anónimo que, con los formatos predeterminados de ASP.NET Core, se serializa en JSON.

La solución activa queda:

```
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return Ok(new { producto.Id, producto.Nombre });
}
```

Por qué no hay vista: `Ok(...)` devuelve los datos serializados; no se ha utilizado `View(...)`. La presencia de un método MVC no obliga a generar HTML. Es una comparación de resultados HTTP, no la introducción de un módulo completo de Web API.

Ejecutar, probar y razonar

Guardar, compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Desde el navegador o con `curl -i`, probar:

```
http://localhost:5091/catalogo/resumen/2
http://localhost:5091/catalogo/resumen/99
http://localhost:5091/catalogo/resumen/no-numero
```

Resultado esperado de la primera solicitud:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"id":2,"nombre":"Lápiz"}
```

El ejemplo muestra los elementos relevantes de la respuesta, no necesariamente todas las cabeceras del servidor. Las otras dos URLs deben devolver **404**, aunque por motivos distintos: ausencia de producto y ausencia de coincidencia con la restricción de ruta `id:int`.

Comprobación adicional: la URL `/Productos/Detalle/2` sigue produciendo HTML y `/Productos/Estado` sigue devolviendo texto. No hemos modificado su comportamiento.

Preguntas de consolidación y solución

¿Qué cambió respecto a `Detalle`? La selección de un resultado `Ok(objeto)` en lugar de `View(producto)`. El producto se consulta de la misma fuente de datos, pero se presenta como JSON.

¿Cuándo se ejecuta `Resumen`? Cuando llega una solicitud GET a la plantilla de ruta por atributos con un segmento `id` entero.

¿Se ejecuta la acción si escribimos `no-numero`? No, porque la ruta del reto incluye la restricción `{id:int}`. El resultado esperado es un 404 de no coincidencia de ruta.

¿Dónde está el código del reto? En este documento y en `PROYECTO/1.4/CursoWeb.Mvc/Controllers/ProductosController.cs`, delimitado por `RETO 1.4`. No se crea una solución ajena al proyecto; el alumno activa la variante incluida.

¿Cómo se prueba automáticamente? Mediante el estado `reto` de `M01_MVC/TESTS/1.4_e2e.sh`, que quita los delimitadores en una copia temporal, compila, arranca Kestrel y comprueba códigos y contenido JSON.

E. Matriz de resultados de toda P1.4

Estado	Bloques activados	Resultado observable
Base del punto 1.4	Ninguno nuevo	`/Productos` 200 y listado; `/Productos/Detalle/2` 404
Acción de estado	1.4.2	`/Productos/Estado` 200 con `text/plain`
Detalle	1.4.2 + 1.4.3	`/Productos/Detalle/2` 200, `/99` 404 y `/0` 400
Ruta por atributos	1.4.2–1.4.4	`/catalogo/ficha/2` 200 y segmento no entero 404
Enlaces de listado	1.4.2–1.4.5	Tres enlaces «Ver detalle» hacia las fichas
Reto	Pasos anteriores + RETO 1.4	`/catalogo/resumen/2` 200 JSON y casos inexistentes 404

Una URL que retorna 404 en el estado base puede retornar 200 después de activar su acción. La diferencia es **el código habilitado**, no el navegador.

F. Pruebas automáticas con servidor HTTP real

Detener la aplicación manual, volver a la **raíz del repositorio** y ejecutar:

```
bash M01_MVC/TESTS/1.4_e2e.sh
```

El script construye y arranca una aplicación en cada estado. Para evitar modificar el proyecto del alumno, crea copias temporales donde elimina únicamente los comentarios de las fases ya habilitadas. Comprueba también el listado de tres productos, la navegación, la hoja CSS y la guía pública de capítulos anteriores.

En los nuevos recorridos comprueba `GET /Productos/Estado`, `/Productos/Detalle/id`, `/catalogo/ficha/id` y `/catalogo/resumen/id`, con sus resultados 200/400/404. El reto recibe JSON real y se comprueba que contenga `id=2` y `nombre=Lápiz`.

Alcance correcto de la prueba: verifica el recorrido HTTP y el HTML resultante en un servidor Kestrel real; no reemplaza la experiencia de pulsar los enlaces en un navegador con interfaz gráfica ni constituye una prueba de seguridad de producción.

G. Problemas frecuentes y soluciones

La aplicación no compila al activar una acción. Revisa que has quitado los dos delimitadores `/* ... */` del bloque y que no has añadido o borrado llaves accidentalmente. Compila después de cada paso para acotar el error.

`/Productos/Estado` sigue devolviendo 404. Verifica que la acción `Estado` está activa y que estás ejecutando `PROYECTO/1.4`, no la copia anterior de 1.3. Un proceso previo podría estar usando el puerto 5091.

`/Productos/Detalle/2` devuelve 404 aunque el id existe. Comprueba que activaste la acción `Detalle`, que la URL usa la ruta convencional correcta y que la lista contiene el id 2.

Un detalle con id 0 devuelve 400. Es el comportamiento correcto del ejercicio: `Detalle(int id)` tiene una comprobación que rechaza valores menores o iguales que cero. No debemos cambiarla para forzar una respuesta 200.

`/catalogo/ficha/2` devuelve 404. La acción `Ficha` debe estar activa y llevar el atributo `[HttpGet("/catalogo/ficha/{id:int}")]`. No basta con agregar una vista llamada Ficha; aquí reutilizamos explícitamente la vista Detalle.

`/Productos/Ficha/2` no funciona. La acción `Ficha` ha definido una URL absoluta por atributos, no la URL convencional que corresponde a su nombre. La petición correcta es `/catalogo/ficha/2`.

Los enlaces de detalle no aparecen. Revisar que se han quitado ambos delimitadores del comentario Razor `PASO 1.4.5` dentro del `@foreach` de `Views/Productos/Index.cshtml`. Una acción que funciona no genera por sí misma enlaces en el listado.

El reto devuelve HTML en vez de JSON. Comprueba que utilizas `Ok(new { ... })` y no `View(producto)`. Inspecciona la cabecera `Content-Type`; en el ejercicio esperamos un formato JSON.

En la respuesta JSON los nombres aparecen en minúscula inicial. Es la convención de serialización habitual, no un cambio del modelo C#. La propiedad `Id` de C# puede aparecer como `id` en JSON.

H. Cierre y continuidad

Al terminar, el alumno debe poder explicar **qué acción MVC se ejecuta para cada URL, cómo obtiene el id y por qué devuelve un resultado HTTP concreto**. El listado sigue funcionando y permite navegar a fichas individuales. Se han utilizado el mismo modelo y la misma vista para distintos recorridos sin una base de datos.

El reto demuestra, mediante código real comentado y comprobable, que un controlador MVC también puede producir JSON. Todos los fragmentos que se han pedido ejecutar existen en el proyecto del capítulo.

Conserva este estado funcional. En el punto **1.5** profundizaremos en las vistas Razor, modelos fuertemente tipados, layouts y vistas parciales, utilizando las fichas que acabamos de construir. Aún no añadiremos formularios ni persistencia: son actividades de M02.

P1.5. Vistas Razor tipadas, layouts y vistas parciales

A. Objetivos y estado inicial

Objetivo: reconocer los elementos de la sintaxis Razor que usamos para construir HTML, distinguir el modelo de una vista de lista del modelo de una ficha, ampliar un layout mediante una sección opcional y reutilizar una vista parcial tipada para mostrar productos en el listado y en la ficha individual.

Reto con código: incorporar en la parcial una etiqueta visual que aparezca cuando el precio sea inferior a tres unidades. El reto está preparado como bloque Razor comentado dentro del proyecto y debe verificarse tanto en el listado como en el detalle. Aquí se proporciona enunciado, solución, explicación y comandos exactos.

Teoría relacionada: capítulo 1.5 del Markdown teórico de M01; conviene tener abiertos los apartados sobre `@model`, expresiones, `@if`, `@foreach`, `@section`, `@RenderBody()` y el Partial Tag Helper.

Tiempo orientativo de aula: 20 minutos de teoría y 50 de práctica incluyendo reto. Las explicaciones desarrolladas permiten estudio individual; las comprobaciones automáticas no consumen tiempo de aula.

Estado inicial: el proyecto del capítulo **1.5** contiene las páginas completas de P1.4: listado con tres enlaces de detalle, acción `Detalle(int id)`, acción `Ficha(int id)` por atributos y la acción de texto `Estado()`. El reto opcional de P1.4 sigue comentado. Lo nuevo de P1.5 está preparado en archivos Razor con comentarios que el alumno irá retirando.

B. Preparación

El proyecto se encuentra en:

```
M01_MVC/PROYECTO/1.5/CursoWeb.Mvc/
├── Controllers/
│   ├── HomeController.cs
│   └── ProductosController.cs
├── Models/
│   └── Producto.cs
├── Views/
│   ├── _ViewStart.cshtml
│   ├── _ViewImports.cshtml
│   ├── Productos/
│   │   ├── Index.cshtml
│   │   └── Detalle.cshtml
│   └── Shared/
│       ├── _Layout.cshtml
│       └── _TarjetaProducto.cshtml
├── wwwroot/
│   ├── css/
│   │   ├── site.css
│   │   └── catalogo.css
│   └── recursos/
└── CursoWeb.Mvc.csproj
```

No editar el proyecto 1.4 para esta práctica: el de 1.5 conserva la evolución anterior y permite activar los siguientes pasos sin reconstruir otras versiones. El archivo parcial `_TarjetaProducto.cshtml` y sus estilos **ya existen**, aunque inicialmente las vistas todavía no lo incluyen.

Abrir `CursoWeb.Mvc.csproj` en Visual Studio, o ejecutar desde la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.5/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

En el navegador comprobar:

```
http://localhost:5091/
http://localhost:5091/Productos
http://localhost:5091/Productos/Detalle/2
http://localhost:5091/catalogo/ficha/2
```

La portada, el catálogo y las dos rutas de detalle deben funcionar. `/Productos` muestra Cuaderno, Lápiz y Carpeta con enlaces «Ver detalle», pero todavía **no** muestra las tarjetas adicionales ni el mensaje de catálogo completo.

Detener la ejecución antes de modificar los bloques. En los pasos siguientes puedes conservar la misma terminal y lanzar nuevamente `dotnet run --no-launch-profile --urls http://localhost:5091` después de compilar.

C. Ejercicios guiados

Paso 1.5.1 — Identificar el tipo de modelo y el origen del título de página

Abrir `Views/Productos/Index.cshtml`. Al principio del archivo observarás:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
@{
    ViewData["Title"] = "Catálogo de productos";
}
```

La directiva `@model` informa al motor Razor de que recibimos **una lista** de `Producto`, o un valor nulo si no se ha proporcionado. El bloque `@{ ... }` ejecuta C# para establecer un título; no dibuja por sí mismo una etiqueta HTML.

Más abajo, la vista tiene una condición `@if (Model is null)`. Cuando no hay modelo presenta un mensaje, y cuando sí lo hay escribe `@Model.Count` y recorre la lista con `@foreach (var producto in Model)`.

Ahora abrir `Views/Productos/Detalle.cshtml`. Su directiva es diferente:

```
@model CursoWeb.Mvc.Models.Producto
@{
    ViewData["Title"] = "Detalle del producto";
}
```

La ficha recibe **un solo producto**. Por eso consulta `@Model.Id`, `@Model.Nombre` y `@Model.Precio` sin recorrer una lista.

Finalmente abrir `Views/Shared/_Layout.cshtml` y localizar:

```
<title>@ViewData["Title"] - Curso Web .NET 8</title>
...
@RenderBody()
```

La etiqueta `<title>` de cada respuesta combina el valor de `ViewData` de la vista con el texto común. `@RenderBody()` establece dónde se inserta el cuerpo de la vista dentro del documento principal.

Comprobación sin editar nada: abre `/Productos` y `/Productos/Detalle/2` y compara los títulos de las pestañas. Después utiliza F12 para inspeccionar el HTML y localizar las etiquetas `<h1>`, `<title>` y los nombres de los productos. En la respuesta HTML no se debe encontrar código ejecutable `@Model.Nombre`: Razor ya lo ha evaluado.

Resultado del paso: identificas dos contratos de modelo diferentes y una responsabilidad del layout. No se requiere descomentar bloques todavía.

Paso 1.5.2 — Incorporar una condición Razor de presentación

Abrir `Views/Productos/Index.cshtml` y localizar dentro del bloque `else` el fragmento comentado:

```
@* PASO 1.5.2 (inicio)
@if (Model.Count >= 3)
{
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
PASO 1.5.2 (fin) *@
```

Acción: eliminar únicamente las dos líneas delimitadoras del comentario Razor que contienen `PASO 1.5.2`. Mantener la instrucción `@if` y el elemento HTML. El código activo queda:

```
@if (Model.Count >= 3)
{
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
```

Explicación: `Model.Count` obtiene la cantidad de productos que entregó el controlador. La condición compara la cantidad con 3 y decide si incluye el párrafo en el HTML final. No modifica la lista ni crea productos nuevos. El bloque está dentro del `else` que garantiza que `Model` no es nulo.

Compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `/Productos`. Debe aparecer el mensaje **El catálogo contiene al menos tres productos**, junto al contador de tres productos y la lista existente. La ficha `/Productos/Detalle/2` continúa igual, porque su vista es distinta.

Comprobación: en F12 → Red/Network → respuesta de `/Productos`, buscar `id="catalogo-completo"`. Este elemento no estaba presente en el estado inicial; aparece exclusivamente después de activar la condición.

Detener la ejecución para pasar al siguiente ejercicio.

Paso 1.5.3 — Extender el layout mediante una sección Razor opcional

Este paso requiere **dos cambios coordinados**. En uno, el layout establece dónde mostrará notas adicionales; en el otro, la vista de detalle define qué texto debe ocupar esa zona.

Primer archivo: **`Views/Shared/_Layout.cshtml`.** Localizar:

```
@* PASO 1.5.3 (inicio)
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
PASO 1.5.3 (fin) *@
```

Quitar solo las líneas delimitadoras del comentario. El bloque activo queda:

```
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
```

Qué hace: `RenderSectionAsync("NotasPagina", required: false)` representa una sección de nombre `NotasPagina` si la vista que se está renderizando la ha definido. Como la sección es **opcional**, no se exige a todas las páginas del sitio que la declaren.

Segundo archivo: **`Views/Productos/Detalle.cshtml`.** Localizar el bloque:

```
@* PASO 1.5.3 (inicio)
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
PASO 1.5.3 (fin) *@
```

Retirar sus dos delimitadores y conservar:

```
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
```

Qué hace: la página de detalle define el contenido que el layout insertará en su zona de notas. Este bloque no genera automáticamente un segundo documento HTML; proporciona un fragmento que el layout representa en la posición elegida.

Importante: en esta práctica, **activa ambos bloques antes de compilar**. Una sección definida por una vista debe ser tratada por su layout; no queremos dejar una sección definida pero sin destino.

Compilar, iniciar y visitar `/Productos/Detalle/2`. Debe seguir mostrando la ficha de Lápiz y, fuera de su cuerpo principal, el mensaje **Detalle construido con una vista Razor tipada**.

A continuación visita `/Productos` y `/`. Ambas deben funcionar sin definir la sección, porque el layout utiliza `required: false`. Es normal que la estructura externa del layout exista aunque una página no aporte contenido a dicha zona.

Resultado de aprendizaje: has observado la diferencia entre una vista concreta y un diseño compartido, y has visto cómo una sección opcional permite personalizar una región de ese diseño.

Detener el servidor antes de continuar.

Paso 1.5.4 — Representar una vista parcial por cada producto del catálogo

El proyecto incorpora una **vista parcial tipada** en `Views/Shared/_TarjetaProducto.cshtml`. Abre el archivo y localiza:

```
@model CursoWeb.Mvc.Models.Producto

<article class="tarjeta-producto" data-producto-id="@Model.Id">
    <h3>@Model.Nombre</h3>
    <p class="tarjeta-precio">Precio: @Model.Precio.ToString("0.00") €</p>
    <a href="/Productos/Detalle/@Model.Id">Ver detalle de @Model.Nombre</a>
</article>
```

En el archivo real hay comentarios docentes y un bloque del reto comentado dentro del artículo. **No actives todavía el reto**. La parcial ya contiene todo lo necesario para mostrar una tarjeta de un producto.

Qué significa el código: `@model Producto` exige un objeto `Producto` individual; `@Model.Nombre` y `@Model.Precio` generan texto; el atributo `data-producto-id` permite observar qué producto se ha pasado. El enlace conduce a su detalle. No hay lógica de acceso a datos ni acciones HTTP en la parcial.

Abre ahora `Views/Productos/Index.cshtml` y localiza:

```
@* PASO 1.5.4 (inicio)
<section id="tarjetas-catalogo" aria-label="Tarjetas reutilizables de productos">
    <h2>Vista de tarjetas</h2>
    @foreach (var producto in Model)
    {
        <partial name="_TarjetaProducto" model="producto" />
    }
</section>
PASO 1.5.4 (fin) *@
```

Acción: eliminar las dos líneas delimitadoras del comentario `PASO 1.5.4`, manteniendo las demás intactas.

Dentro del `foreach`, la variable `producto` se refiere a un objeto de la colección. El Partial Tag Helper `<partial name="_TarjetaProducto" model="producto" />` entrega ese objeto a `_TarjetaProducto.cshtml`, que lo recibe como su `Model`.

¿Por qué funciona? Porque en `Views/_ViewImports.cshtml` ya existe el registro de Tag Helpers de MVC:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Sin esa directiva, la sintaxis del elemento `<partial>` no se procesaría como esperamos. No hace falta instalar otra biblioteca para el curso.

Compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `/Productos`. Debajo de la lista clásica debe aparecer la sección **Vista de tarjetas**, con **tres tarjetas**: Cuaderno, Lápiz y Carpeta. Cada tarjeta muestra nombre, precio y un enlace de detalle. El listado anterior continúa visible para comparar las dos formas de representación.

Comprobación técnica: en F12 → Red → Response, localizar `id="tarjetas-catalogo"` y exactamente **tres** elementos con `class="tarjeta-producto"`. No se han generado tres peticiones HTTP nuevas para las tarjetas: son fragmentos de una única respuesta HTML.

Paso 1.5.5 — Reutilizar la misma parcial en la vista de detalle

Detener la aplicación y abrir `Views/Productos/Detalle.cshtml`. Localizar el bloque:

```
@* PASO 1.5.5 (inicio)
<section id="tarjeta-detalle" aria-label="Ficha presentada como tarjeta">
    <h2>Tarjeta reutilizada en el detalle</h2>
    <partial name="_TarjetaProducto" model="Model" />
</section>
PASO 1.5.5 (fin) *@
```

Acción: retirar los dos delimitadores del comentario y dejar activo el elemento `<section>` completo.

Diferencia importante respecto al paso 1.5.4: en la vista del catálogo pasábamos `model="producto"` porque había un `foreach` sobre una colección. En la vista de detalle, el `Model` ya es un solo `Producto`, por lo que utilizamos `model="Model"`.

La parcial `_TarjetaProducto.cshtml` **no se copia** ni se duplica. Se encuentra en `Views/Shared` y el motor Razor la utiliza desde los dos archivos de vistas.

Compilar y ejecutar de nuevo. Visita `/Productos/Detalle/2`: bajo los datos de la ficha debe aparecer una tarjeta adicional de **Lápiz**, con su nombre, precio y enlace. Visita también `/Productos/Detalle/1`: debe aparecer una tarjeta de **Cuaderno**.

Regresar a `/Productos`: sus tres tarjetas continúan presentes y no han cambiado de estructura.

Conclusión del paso: el mismo archivo de parcial puede renderizarse muchas veces con modelos distintos, en páginas diferentes, sin duplicar marcado. Esta es la principal ventaja de la reutilización que se quiere enseñar en P1.5.

D. Reto 1.5 — Destacar productos económicos reutilizando una única parcial

Enunciado

El catálogo de prueba tiene estos precios:

Producto	Precio
Cuaderno	4,50 €
Lápiz	1,20 €
Carpeta	3,75 €

Se pide que las **tarjetas de productos con precio inferior a 3 €** muestren el texto **Precio económico**. La condición debe escribirse **solo en la vista parcial** y afectar automáticamente tanto a las tarjetas del listado como a la tarjeta de la ficha de detalle.

Resultado esperado: Lápiz muestra la etiqueta; Cuaderno y Carpeta no. No cambies `ProductosController.cs`, los modelos ni los archivos de las vistas que incluyen la parcial.

Código del reto incluido en el proyecto

Abrir `Views/Shared/_TarjetaProducto.cshtml` y localizar este fragmento incluido inicialmente como comentario Razor dentro del artículo:

```
@* RETO 1.5 (inicio)
@if (Model.Precio < 3m)
{
    <strong class="etiqueta-economico">Precio económico</strong>
}
RETO 1.5 (fin) *@
```

Acción 1: eliminar únicamente las líneas que contienen `@* RETO 1.5 (inicio)` y `RETO 1.5 (fin) *@`, conservando intacto el contenido intermedio. La solución activa es:

```
@if (Model.Precio < 3m)
{
```

```
    <strong class="etiqueta-economico">Precio económico</strong>
}
```

Explicación de cada parte: `@if` ejecuta una condición de presentación; `Model.Precio` consulta el precio del objeto actual que la vista parcial recibió; `3m` es un literal decimal de C#; `<strong>` genera un elemento HTML solo cuando el precio satisface la condición. La clase `etiqueta-economico` ya dispone de un estilo básico en `wwwroot/css/site.css`.

Acción 2: no modificar ningún otro archivo. La tarjeta parcial se utiliza tanto dentro de las tres tarjetas del catálogo como dentro de la ficha de un producto. Por tanto, una única modificación debe observarse en ambas ubicaciones.

Acción 3: compilar e iniciar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Comprobar primero `http://localhost:5091/Productos`: debe aparecer **una sola etiqueta Precio económico** en la tarjeta de Lápiz. Las tarjetas de Cuaderno y Carpeta no deben recibir esa etiqueta.

Después visitar `http://localhost:5091/Productos/Detalle/2`: la tarjeta reutilizada de Lápiz debe mostrar también la etiqueta. Visitar `http://localhost:5091/Productos/Detalle/1`: la ficha de Cuaderno no debe mostrarla.

Solución razonada y preguntas

¿Por qué solo aparece una etiqueta en el catálogo? Porque se recorre una colección de tres productos, pero solo Lápiz tiene `Precio < 3m`.

¿Por qué el detalle de Lápiz cambia sin editar `Detalle.cshtml`? Porque `Detalle.cshtml` representa la misma parcial, pasándole el objeto de Lápiz. La regla se evalúa dentro de la parcial y produce el mismo resultado en ambos lugares.

¿Qué sucedería si copiáramos la condición en cada vista, en lugar de tenerla en la parcial? Tendríamos dos sitios donde mantener una misma regla de presentación; sería fácil modificar uno y olvidar el otro.

¿Hemos calculado un descuento real? No. La etiqueta es una indicación visual de ejemplo. No cambia precios ni aplica una política comercial.

¿Dónde comprobar el código? En este Markdown y en `PROYECTO/1.5/CursoWeb.Mvc/Views/Shared/_TarjetaProducto.cshtml`. El proyecto ya contiene el código; el alumno no necesita inventarlo ni buscar una solución externa.

Verificación del reto

Desde la **raíz del repositorio**, con la instancia manual detenida, ejecutar:

```
bash M01_MVC/TESTS/1.5_e2e.sh
```

El escenario `reto` activa automáticamente la condición en una copia temporal junto con los pasos 1.5.2–1.5.5. Compila, inicia Kestrel y verifica que existe **una** etiqueta en el catálogo, **una** en `Detalle/2` y **ninguna** en `Detalle/1`. También vuelve a comprobar que las páginas y rutas anteriores siguen funcionando.

E. Tabla de comprobación de la práctica

Estado	Código activado	Qué debe poder comprobarse
Base	Ningún bloque nuevo	Listado y detalle de P1.4 funcionales
Condición Razor	1.5.2	Mensaje «El catálogo contiene al menos tres productos»
Sección del layout	1.5.2 + ambos bloques 1.5.3	Nota adicional en ficha; listado y portada funcionan sin sección
Tarjetas en listado	Anteriores + 1.5.4	Tres tarjetas procedentes de `_TarjetaProducto.cshtml`
Tarjeta en detalle	Anteriores + 1.5.5	Una tarjeta de producto en cada ficha
Reto	Todo lo anterior + RETO 1.5	Lápiz se etiqueta como económico en catálogo y detalle; los otros productos no

En cada etapa se debe confirmar al menos una vez que el código compila y el HTML esperado llega al navegador. Las pruebas automatizadas reconstruyen las seis etapas completas con una aplicación HTTP real.

F. Problemas frecuentes y soluciones

La vista no encuentra `_TarjetaProducto`. Comprobar que el archivo está en `Views/Shared/_TarjetaProducto.cshtml`, que se escribe su nombre de forma correcta en el `<partial>` y que se está ejecutando el proyecto de 1.5.

Aparece literalmente una etiqueta `<partial>` en el HTML sin las tarjetas. Revisar que `Views/_ViewImports.cshtml` conserva `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`. El Tag Helper necesita este registro.

Al activar el paso 1.5.3 se obtiene un error por secciones no representadas. Es necesario activar ambos bloques del mismo paso: `RenderSectionAsync` en el layout y `@section` en la vista de detalle. No hay que activar únicamente la mitad correspondiente a una vista.

La portada o el listado falla después de añadir la sección. Comprobar que en el layout se escribió `required: false`. Una sección obligatoria exigiría que todas las vistas del sitio la definieran.

El listado dice que no hay modelo. En el proyecto de 1.5 el controlador `Index` ya envía una colección. Verificar que se está ejecutando la carpeta correcta y no un proyecto anterior.

La vista parcial se genera tres veces dentro del detalle. Revisar que en `Detalle.cshtml` se usa un único `<partial model="Model" />` y no un bucle, porque `Model` ya es un objeto individual.

No aparece la etiqueta del reto. Revisar que se han quitado ambas líneas del comentario Razor del bloque `RETO 1.5`, que está activo el paso 1.5.4 o 1.5.5 que incluye la parcial y que se ha reconstruido la aplicación.

La etiqueta también aparece en Cuaderno. Comprobar que la comparación es estrictamente `Model.Precio < 3m` y que no se cambió el precio del producto.

Se intenta colocar el contenido de `@section` dentro de la parcial. Las secciones están asociadas a vistas completas y sus layouts. El ejercicio define `NotasPagina` en `Detalle.cshtml`, no en `_TarjetaProducto.cshtml`.

La lista muestra caracteres escapados al inspeccionar el HTML. Razor puede codificar caracteres como entidades HTML. El navegador presenta el texto correcto; esto no significa que se haya modificado el nombre del producto.

G. Verificación final de M01 y preparación de M02

Tras completar el reto, recorre el sitio: portada, listado con tres productos, ficha de Lápiz, ficha de Cuaderno, ruta por atributos y recurso CSS. En el listado comprueba la condición de tres productos, las tarjetas y los enlaces; en el detalle comprueba la sección opcional del layout y la tarjeta reutilizada.

Cada funcionalidad debe poder asociarse a una pieza concreta: datos en el controlador, contrato `Producto` en el modelo, presentación en vistas Razor, estructura común en el layout y marcado repetido en la parcial.

El resultado de M01 es un catálogo consultable y navegable, todavía con datos en memoria. No hemos desarrollado todavía creación o edición de productos. En M02 añadiremos formularios, binding de GET/POST, validación, Tag Helpers de formularios y persistencia EF Core. Se mantiene así el alcance de las 30 horas acordadas.

Para continuar: conserva el proyecto 1.5 como referencia del catálogo MVC desarrollado. En M02 lo ampliaremos con operaciones de creación, edición y eliminación, manteniendo los conceptos de separación de responsabilidades, rutas y vistas que has practicado.